

Praktikum 3: *Semantische Analyse*

Ausgabe:

Dienstag, 3. November 2025

Abgabe:

siehe Moodle

Inhaltsübersicht:

Überblick	1
Erweitern der Klasse Sym.java	3
Namensanalyse	4
Mehrfach- und nicht deklarierte Bezeichner	4
Die Types Klasse	5
Test auf korrekte Return-Anweisungen	6
Hinweise zur Vorgehensweise	6
Was ist abzugeben?	7
Appendix 1: Allgemeine Semantische Regeln für Simple	8
Semantik der switch-Anweisung	8

Überblick

Wir sind in der letzten Phase des Frontend wieder einen Schritt näher an der Generierung des realen Codes! Ihre Aufgabe besteht dieses Mal, eine semantische Analyse zu schreiben. Die Aufgabenbeschreibung ist dieses Mal recht umfangreich, Sie werden jedoch feststellen, dass die semantische Analyse selbst relativ einfach zu realisieren ist und nicht besonders umfangreich ist.

Wer braucht einen Semantik-Checker? Programmierer, die Fehler machen! Zu den Fehlern, die die Semantische Analyse finden kann, gehört eine vergessene Variablendeklaration oder falsche Verwendung von Datentypen. Weiterhin benötigt der Codegenerator die Semantische Analyse. Dies ist erforderlich, da man für semantisch unkorrekten Quellcode auch keinen

Maschinencode generieren kann. Es wäre zum Beispiel nicht klar, welche MIPS-Anweisungen, nötig sind, um eine Gleitkommazahl mit einer Zeichenfolge zu vergleichen.

Für diese Aufgabe machen Sie folgendes:

Schreiben Sie eine statisch semantische Analyse im “two pass” Verfahren, sprich Sie durchlaufen das Simple-Programm, das nun als AST vorliegt, zweimal, um es zu analysieren.

- Erstellen Sie dazu eine **Symboltabelle**. Sie bekommen dazu eine Vorabversion, die Sie ergänzen müssen.
- Um die semantische Information von Bezeichnern (IDs) zu speichern, **ändern Sie die `IdNode`-Klasse in `ast.java`** (indem Sie ein neues Feld einfügen und die Dekompilierungsmethode ändern).
- **Schreiben Sie ein neues Hauptprogramm, `P4.java`**, eine Erweiterung von `P3.java` — das alte `P3` brauchen Sie dann nicht mehr.
- **Aktualisieren Sie das `Makefile` um `P4.java`** in die Kompilierung aufzunehmen.
- Schreiben Sie drei Testeingaben: `name_errors.sim`, `type_errors.sim` und `test.sim`, um Ihren neuen Code zu testen. Die erste sollte Fehler bezüglich Bezeichnern testen, die zweite allgemeine Typfehler und die letzte weitere, korrekte Eingaben.

Die Dateien bekommen Sie wieder als Docker-Container, in dem alle notwendige Software bereits installiert ist:

```
docker run -it markusmock/ib610:typechecker /bin/bash
```

, um den Container auszuführen und darin eine Shell zu starten.
Die Dateien befinden sich im Verzeichnis `ib610/typechecker`.

Kopieren Sie Ihre Parserimplementierung in dieses Verzeichnis, um den Parser mit der semantischen Analyse zu erweitern. Wie in den Praktika 1 und 2 beschrieben, ist es empfehlenswert, Ihr Host-Verzeichnis mittels `-v` im Container zugänglich zu machen.

Die beiden Durchläufe der statischen semantischen Analyse, die Sie schreiben werden, sind die (1) Namensanalyse und die (2) Typanalyse. Die Namensanalyse führt die folgenden Aufgaben aus (die semantischen Regeln von Simple finden Sie im Anhang dieser Beschreibung).

1. **Erstellen Sie Symboltabellen.** Sie können entweder den Ansatz "Liste von Hash-Tabellen" verwenden oder einfach eine Symboltabelle für den Klassennamen verwenden, eine Symboltabelle für alle Klassenvariablen und Methodennamen und für jede Methode eine lokale Tabelle für die Parameter und lokalen Variablen der Methode. Für geschachtelte Blöcke benötigen Sie natürlich auch separate Einträge.
2. **Finden Sie mehrfach deklarierte Namen und Verwendungen von nicht deklarierten Namen.** Im Gegensatz zu Java erlaubt die Sprache Simple nicht, dass ein und derselbe Name *mehr als einmal pro Gültigkeitsbereich* deklariert wird, auch wenn es sich um unterschiedliche "Arten" handelt, und es ist erforderlich, dass alle Namen deklariert werden, bevor sie verwendet werden. Ein Name kann aber selbstverständlich in einem anderen Gültigkeitsbereich wieder verwendet werden.
3. **Aktualisieren Sie die `IdNodes`** des AST, um Verweise auf die entsprechenden Symboltabelleneinträge (d. h. auf Objekte vom Typ `Sym`) aufzunehmen.

Die Typprüfung ermittelt den Typ jedes Ausdrucks, der im Abstract-Syntax-Baum dargestellt ist, und verwendet diese Informationen, um Typfehler zu identifizieren, wie unten beschrieben.

Das Hauptprogramm, `P4.java`, ähnelt `P3.java`, mit der Ausnahme, dass nach dem Parsen, wenn keine Syntaxfehler vorliegen, der Name-Analyzer aufgerufen wird, bevor der Decompiler aufgerufen wird. Wenn nach dem Aufruf des Decompilers bis dahin keine Fehler aufgetreten sind, wird die Typprüfung aufgerufen. Das Aufrufen der Namensprüfung und der Typprüfung bedeutet, dass die entsprechenden Methoden des AST-Knotens aufgerufen werden, der die Wurzel des vom Parser erstellten Baums ist.

Der Dekompiler unterscheidet sich geringfügig von der für Praktikum 3 geschriebenen Version: Beim Dekompilieren eines `IdNode` wird der Typ des Knotens (in Klammern) nach dem Namen gedruckt. Auf diese Weise können Sie feststellen, ob Ihre Namensprüfung ordnungsgemäß funktioniert.

Sie werden auch drei Eingabedateien schreiben, um Ihren Code zu testen: `nameErrors.sim` sollte Code mit Fehlern enthalten, die von der Namensanalyse erkannt wurden, `typeErrors.sim` sollte Code mit Fehlern enthalten, die von der Typprüfung erkannt wurden, und `test.sim` sollte Code ohne Fehler enthalten .

Sie werden nach der Korrektheit Ihres Codes beurteilt und danach, wie gründlich Ihre Eingabedateien Ihren Code testen.

In Moodle finden Sie wieder Hilfsdateien, im einzelnen:

- `ast.java` und `simple.cup`: verwenden Sie Ihre Lösung aus dem Parser-Praktikum weiter.
- `Types.java`: Die `Types`-Klasse kann von Ihrer Namensanalyse und Ihrem Type-Checker verwendet werden (siehe unten).
- `SymbolTable.java`: Sie müssen diese Datei ergänzen. Möglicherweise müssen Sie diese Symboltabellenklasse an die Anforderungen eines Compilers anpassen.

Erweitern der Klasse Sym.java

Ein Symboltabelleneintrag muss den Typ jedes Symbols sowie seinen Namen enthalten (siehe unten für Ratschläge zur Darstellung von Typen). Bei Methodennamen muss der Symboltabelleneintrag auch Informationen über die Anzahl der Parameter und deren Typen enthalten. Dies kann durch eine Liste der Symboltabelleneinträge für die Parameter erreicht werden. Alle diese Informationen werden benötigt, um die Typprüfung zu implementieren. Daher müssen Sie die `Sym`-Klasse ändern, indem Sie einige neue Felder hinzufügen. Möglicherweise möchten Sie auch eine neue Methode hinzufügen: eine `ToString`-Methode oder eine `Print`-Methode oder eine Methode, die den `Sym`-Typ zurückgibt, die beim Dekompilieren eines Bezeichners verwendet werden soll.

Namensanalyse

Sie müssen Ihre Namensanalyse dadurch implementieren, indem Sie geeignete Methoden für die verschiedenen Unterklassen von `ASTNode` schreiben. Einige der Methoden verarbeiten die *Deklarationen* im Programm (prüfen, ob die Namen mehrfach deklariert sind, und falls nicht, fügen Sie entsprechende Symboltabelleneinträge hinzu), und andere verarbeiten die *Anweisungen* im Programm (prüfen, ob jeder Name der in einer Anweisung verwendet wird, deklariert wurde). Die `IdNode`-Klasse benötigt eine Methode jeder Art, also für Deklaration und Anweisungen, da eine ID sowohl in einer Deklaration als auch in einer Anweisung enthalten sein kann. Diese Methoden müssen den `IdNode` aktualisieren, indem sie eine Referenz zum entsprechenden Symboltabelleneintrag hinzufügen. Dies bedeutet, dass Sie der `IdNode`-Klasse ein Feld vom Typ "Sym" hinzufügen müssen.

Mehrfach- und nicht deklarierte Bezeichner

Die Namensanalyse ist dafür verantwortlich, alle Instanzen mehrfach deklarerter Bezeichner zu finden (Bezeichner, die mehr als einmal im selben Bereich deklariert wurden). Das beinhaltet:

- Zwei Klassenvariablen, zwei Methoden oder eine Klassenvariable und eine Methode mit demselben Namen.
- Zwei formale Parameter derselben Methode mit demselben Namen.
- Ein Formalparameter und eine lokale Variable einer Methode mit demselben Namen.
- Zwei lokale Variablen einer Methode mit demselben Namen.

Für jeden mehrfach deklarierten Bezeichner sollte eine Fehlermeldung (siehe unten) ausgegeben werden, und alle außer der ersten Deklaration sollten ignoriert werden (d.h. aktualisieren Sie die Symboltabelle nicht, wenn Sie einen mehrfach deklarierten Bezeichner verarbeiten, und verknüpfen Sie diesen `IdNode` mit keinem Symboltabelleneintrag).

Die Namensanalyse ist auch dafür verantwortlich, alle Fälle von nicht deklarierten Bezeichnern zu finden. Beachten Sie, dass dies Aufrufe von Methoden einschließt, die noch nicht deklariert wurden, und Verwendung von Bezeichnern, die noch nicht deklariert wurden. Für jede Verwendung eines nicht deklarierten Bezeichners sollte eine Fehlermeldung (siehe unten) ausgegeben werden. Es liegt an Ihnen, ob Sie die Verwendung des Klassennamens während der Namensanalyse als nicht deklarierten Bezeichner oder bei der Typprüfung als Typfehler behandeln. Wenn Sie letztere Variante auswählen, benötigen Sie zusätzlich zu unten beschriebenen Fehlern einen weiteren Fehler: "Equality operator applied to a class"

Darüber hinaus ist die Namensanalyse dafür verantwortlich, zu überprüfen, ob das Programm eine Methode namens `main` enthält. Wenn das Programm dies nicht tut, muss eine Fehlermeldung (siehe unten) ausgegeben werden.

Zusammenfassend sollte Ihrer Namensanalyse alle in der Tabelle beschriebenen Fehler finden. Sie muss die angegebene Position des Fehlers melden und **genau die unten angegebene Fehlermeldung** anzeigen: jede Meldung sollte in einer einzelnen Zeile erscheinen, also nicht wie in der folgenden Tabelle formatiert. Fehlermeldungen sollten

dasselbe Format wie im Scanner und Parser haben, d. h., sie sollten unter Verwendung eines Aufrufs von `Errors.fatal` ausgegeben werden.

Semantische Fehler in Simple

Fehlerart	Fehlermeldung	Anzugebende Fehlerposition
Mehr als eine Deklaration im selben Gültigkeitsbereich	Multiply declared identifier	Der erste Buchstabe des Bezeichners der duplizierten Deklaration
Verwendung eines nicht deklarierten Bezeichners	Undeclared identifier	Der erste Buchstabe des nicht deklarierten Bezeichners
Keine main Methode im Program	No main method declared	0:0

Die Types Klasse

Sie benötigen eine Möglichkeit, die verschiedenen Typen der in einem Programm gefundenen Bezeichner und Ausdrücke darzustellen. Es liegt an Ihnen, wie Sie dies tun, aber eine Möglichkeit besteht darin, die Werte in der in `Types.java` definierten Klasse zu verwenden. Beispielsweise würden Sie `int` als Typ des neuen Felds verwenden, das jedem Symboltabelleneintrag hinzugefügt wird, und auch als Typ, der von den Typprüfungsmethoden von `ExpNodes` zurückgegeben wird. Sie würden Folgendes verwenden: `Types.IntType` als Wert, der den `int`-Typ darstellt.

Beachten Sie, dass der Zweck von `Types.ErrorType` darin besteht, zu verhindern, dass ein einzelner Typfehler in einem Ausdruck oder einer Anweisung mehrere Fehlermeldungen auslöst. Beispielsweise sollte jeder der folgenden Ausdrücke nur eine Fehlermeldung verursachen:

```
(true 3) * 4
true && (false || 3)
```

Der für `(true 3)` angegebene Typ sollte `ErrorType` sein, und die Typprüfmethode für den Multiplikationsknoten sollte nicht "Arithmetic operator applied to non-numeric operand" für den ersten Operanden melden. Beachten Sie jedoch, dass das Folgende zwei Fehlermeldungen verursachen sollte, eine für jeden der nicht ganzzahligen Operanden des Plus-Operators:

```
true + "hallo"
```

Um Ihnen bei dieser Aufgabe zu helfen, finden Sie die Beispiel-Eingabedatei `example.simple` mit den entsprechenden Fehlermeldungen in `example-err.txt`

Wichtig: Dies ist kein vollständiger Test der Typprüfung. Er soll Ihnen lediglich helfen, einige der Meldungen zu verstehen, die Sie melden müssen.

Denken Sie auch daran, die Funktionsaufrufe zu prüfen! Insbesondere ob die Anzahl und Typen der Argumente stimmen und die return-Anweisungen typkorrekt sind.

Test auf korrekte Return-Anweisungen

Bei Methoden die Ergebnisse zurückliefern müssen (also nicht void als Ergebnistyp haben) müssen wir sicher stellen, dass das zur Laufzeit auch geschieht. Da Simple jedoch bedingte Anweisungen hat, ist es nicht immer statisch, d.h. zur Übersetzungszeit entscheidbar, ob das auch passieren wird. Deshalb prüfen Sie für solche Methoden oder es mindesten eine return Anweisung gibt (die natürlich zum Ergebnistyp passen muss): z.B. muss eine Funktion vom Typ `int foo()` (mindestens) eine `return` Anweisung haben, die einen `int` Wert zurückliefert.

Im Codegenerator werden wir für non-void Methoden auch eine Laufzeit-Überprüfung einbauen, die sicherstellt, dass die Methode auch durch ein expliziten `return` mit Ergebnis verlassen wird

Hinweise zur Vorgehensweise

Hier sind einige Ratschläge zu verschiedenen Aspekten, die in der Aufgabe auftauchen:

In dieser Aufgabe, können Sie alle gewünschten Änderungen am Code in `ast.java` vornehmen. Beispielsweise kann es hilfreich sein, kleine Änderungen an der Klassenhierarchie vorzunehmen oder einigen Klassen neue Felder und / oder Methoden hinzuzufügen.

Der Tree-Traversal-Code, den Sie für die Dekompilierung geschrieben haben, bietet ein gutes Modell für die Traversals, die Sie für die Namensanalyse und die Typprüfung schreiben müssen. Wahrscheinlich möchten Sie die Methoden zur Namensanalyse und Typprüfung jedoch nicht als abstrakte Methoden der Klasse `ASTNode` deklarieren (wie wir es beim Dekompilieren getan haben). Der Grund dafür ist, dass Sie eine Methode für alle Unterklassen von `ASTNode` definieren müssten. Dies war für die Dekompilierung in Ordnung, da wir für jeden Knoten in der Struktur eine passende Ausgabemethode haben wollten. Für die Typprüfung (als Beispiel) muss der Typprüfungsdurchlauf jedoch nicht alle Teile des Baums besuchen, sodass nicht alle Knotenklassen eine Typprüfungsmethode bereitstellen müssen.

Sie müssen jedoch die Methoden für die Namensanalyse und die Typprüfung als abstrakte Methoden für einige der Klassen deklarieren, die sich in der Vererbungshierarchie weiter unten befinden. Beispielsweise müssen Sie eine abstrakte Symboltabellen-Erstellungsmethode für die `DeclNode`-Klasse deklarieren, da die Methode für die `DeclListNode`-Klasse diese Methode für jeden Knoten in der Liste aufruft.

Es kann nützlich sein, eine Ausgabemethode der Symboltabelle zu schreiben, mit der alle Einträge einschließlich ihrer Typen gedruckt werden. Beachten Sie, dass eine solche Printmethode möglicherweise bereits in der Standard-Java-API verfügbar ist. Fügen Sie einen Aufruf der Printmethode für die Symboltabelle in Ihren Debugging-Code ein: Drucken Sie z. B. jede Symboltabelle nach ihrer Erstellung aus, d.h. fügen Sie der `ASTNode`-Klasse ein

`public static bool DEBUG` Feld hinzu und fügen Sie Code in die entsprechenden Methoden zur Namensanalyse ein, um Symboltabellen drucken, wenn `DEBUG` wahr ist.

Inkrementell können Sie Ihre Semantische Analyse so aufteilen dass Sie erst einen kleinen Teil der Phase der Namensanalyse zum Laufen zu bringen (z. B. das Auffinden eines mehrfach deklarierten Feld-Namen). Anschließend können Sie die `ASTnode`-Unterklassen aufteilen und jeweils die Methoden zur Namensanalyse und Typprüfung für Ihre Untergruppe dieser Klassen implementieren: möglicherweise möchten Sie zunächst jeweils nur einige auswählen, bis Sie eine bessere Vorstellung davon haben, welche die meiste Arbeit erfordern. Das macht auch ein inkrementelles Testen möglich, was anzuraten ist.

Was ist abzugeben?

Um Ihr Programm abzugeben, kopieren Sie alle Dateien, die zum Erstellen und Ausführen Ihres Hauptprogramms notwendig sind, sowie Ihr Makefile und Ihre Test Dateien in ein Verzeichnis, und erzeugen Sie daraus eine .zip Datei (make submit mit Makefile das wir Ihnen geben sollte funktionieren) und laden Sie diese in Moodle hoch **und demonstrieren Sie es im Praktikum dem Praktikumsleiter. Reines Hochladen IST KEINE ERFOLGREICHE ABGABE.**

Appendix 1: Allgemeine Semantische Regeln für Simple

- Simple verwendet “static scoping”, das heißt, Gültigkeitsbereiche sind lexikalisch definiert.
- Alle Namen (Bezeichner) müssen vor der ersten Nutzung deklariert werden.
- Es ist keine Mehrfachdeklaration desselben Namens im selben Gültigkeitsbereich erlaubt, selbst wenn sie für eine andere Art von Namen erfolgt.
- Derselbe Name darf in einem (oder mehreren) geschachtelten Gültigkeitsbereichen verwendet werden.
- Die (formalen) Parameter einer Methode sind im selben Gültigkeitsbereich, wie die lokalen Variablen der Methode.

Diese Vorgaben erleichtern die Überprüfung von semantischen Fehlern wesentlich.

Semantik der switch-Anweisung

Dieser Teil ist optional, insbesondere dann, wenn Sie switch nicht in Ihrem Parser implementiert haben.

In der Grammatik definieren folgende Produktionen die Syntax der switch-Anweisung:

- (1) SWITCH LPAREN exp RPAREN LCURLY switchGroupList RCURLY
- (2) switchGroupList ::= switchGroupList switchGroup
 | /* epsilon */
 ;
- (3) switchGroup ::= switchLabel stmtList
 ;
- (4) switchLabel ::= CASE exp COLON
 | DEFAULT COLON
 ;

In Simple sind nur Ausdrücke von Typ Integer als Ausdrücke in (1) erlaubt; Sie müssen also prüfen, dass exp zu einem Integer Typ evaluiert.

Zusätzlich müssen die Ausdrücke in (4) **konstante** Integer sein. Wollen Sie das exakt lösen, müssen Sie eine Typanalyse für konstante Ausdrücke durchführen, was rekursiv mit folgenden Regeln ganz einfach geht:

- i) IntLitereale sind konstant
- ii) Unäre Operatoren auf konstanten Ausdrücken sind wieder konstant
- iii) Binäre Operatoren produzieren konstante Ausdrücke wenn beide Operatoren konstant sind.

Erlaubte Vereinfachung: Sie können auch ausschließlich IntLitereale als Labels als CASE Ausdrücke zulassen.